Unidad 2. Patrones creacionales: Factory Method, Builder y Singleton

Patrones de Diseño · Facultad de Ingeniería · Notas de clase para uso académico

1. ¿Qué problema resuelven?

Cuando el código cliente crea objetos con new en muchos lugares, queda atado a clases concretas: agregar un tipo nuevo obliga a tocar cada uno de esos lugares. Los patrones creacionales centralizan o encapsulan la creación para que el resto del sistema dependa de abstracciones.

2. Factory Method

Define un método para crear objetos y deja que las subclases o una fábrica decidan qué clase concreta instanciar. El cliente pide "un notificador" sin saber si recibirá uno de correo o de SMS.

```
interface Notificador { void enviar(String mensaje); }

class FabricaNotificadores {
    static Notificador crear(String canal) {
        return switch (canal) {
            case "correo" -> new NotificadorCorreo();
            case "sms" -> new NotificadorSms();
            default -> throw new IllegalArgumentException(canal);
        };
    }
}
```

3. Builder

Separa la construcción de un objeto complejo de su representación. Es útil cuando un constructor tendría muchos parámetros opcionales: en lugar de new Reporte(a, null, null, true, 3), se escribe una cadena legible y el método build() valida el resultado final.

```
Reporte r = Reporte.builder()
        .curso("Bases de Datos I")
        .incluirGraficas(true)
        .corte(2)
        .build();
```

4. Singleton y sus riesgos

Garantiza una única instancia con un punto de acceso global. Es tentador para configuraciones o conexiones, pero introduce estado global, dificulta las pruebas unitarias y oculta dependencias. En aplicaciones con inyección de dependencias (Spring, por ejemplo) se prefiere que el contenedor administre una sola instancia y la inyecte donde se necesite.

5. Cómo elegir

- Muchos tipos intercambiables según un dato de entrada: Factory Method.

- Objetos con muchos campos opcionales o pasos de armado: Builder.
- Un recurso realmente único: primero considera inyección de dependencias; Singleton solo si no hay contenedor.
- Familias de objetos que deben combinar entre sí (botón y ventana del mismo tema): Abstract Factory.

6. Ejercicios de práctica

1. Refactoriza un método con un switch que crea distintos tipos de pago para usar Factory Method.
2. Diseña un Builder para la clase Taller (título, nivel, actividades, duración). ¿Qué valida build()?
3. Explica dos problemas de pruebas que causa un Singleton con estado mutable.
4. ¿Qué principio SOLID ayuda a cumplir Factory Method? Justifica.

Fuentes consultadas

• Gamma, Helm, Johnson y Vlissides (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. Patrones creacionales.
• Refactoring.Guru. Patrones creacionales. refactoring.guru/es/design-patterns/creational-patterns
• Bloch, J. (2018). Effective Java, 3.ª ed. Addison-Wesley. Ítems 1-3.
Texto original redactado para el curso a partir de las fuentes citadas.